Thadomal Shahani Engineering College, Bandra

Department of Artificial Intelligence & Data Science

Name: Tanay Malvankar				Subject: ASD&D
T.E. - T1	Batch: T13	Sem: V	Roll No: 58	Date: 31/07/26

Experiment 3

Aim: To deploy multi-service applications using Docker Compose.

Lab Outcome: Mapped LO4

LO4: Containerize, orchestrate and deploy applications using Docker, Docker Compose, Kubernetes and cloud infrastructure tools.

Theory:

Docker Compose

Docker Compose is a tool used to define, configure, and manage multi-container Docker applications. Instead of creating and running each container individually, Docker Compose uses a single configuration file named compose.yml that contains the definitions of all services, networks, volumes, and other application settings.

Docker Compose enables multiple containers to be deployed, monitored, and managed together using a single command. It automatically creates the required network for communication between containers and supports persistent storage

through Docker volumes.

In this experiment, a multi-service application consisting of the following services was deployed:

- Web Server – Nginx
- Application Server – Node.js
- Database Server – MySQL
- Cache Server – Redis

Step-by-Step Explanation of Docker Compose Configuration

Step 1: Define Services

services:

Explanation:

The services section is used to define all the containers that Docker Compose will create and manage.

Step 2: Configure the Web Service

web:

Creates a service named web.

image: nginx:latest

Downloads and uses the latest Nginx image from Docker Hub.

container_name: nginx-web

Assigns the container name nginx-web.

ports:

- "8080:80"

Maps port 8080 of the host system to port 80 inside the container, allowing the web server to be accessed through:

http://localhost:8080

depends_on:

- app

Specifies that the web service depends on the application service.

networks:

- app-network

Connects the web service to the custom Docker network.

Step 3: Configure the Application Service

app:

Creates the backend application service.

image: node:18-alpine

Uses the lightweight Node.js 18 Alpine image.

container_name: node-app

Assigns the container name node-app.

command: sh -c "while true; do echo 'Node.js Application Running'; sleep 30; done"

Executes a continuous command that keeps the container running and simulates a backend application.

networks:

- app-network

Connects the application service to the Docker network.

Step 4: Configure the Database Service

db:

Creates the MySQL database service.

image: mysql:8.0

Uses the MySQL 8.0 Docker image.

container_name: mysql-db

Assigns the container name mysql-db.

environment:

Defines environment variables.

MYSQL_ROOT_PASSWORD: root123

Sets the MySQL root password.

MYSQL_DATABASE: mydb

Automatically creates a database named mydb.

volumes:

  - mysql_data:/var/lib/mysql

Stores MySQL data in the mysql_data Docker volume for persistent storage.

networks:

  - app-network

Connects the database service to the Docker network.

Step 5: Configure the Cache Service

cache:

Creates the Redis cache service.

image: redis:latest

Uses the latest Redis Docker image.

container_name: redis-cache

Assigns the container name redis-cache.

networks:

- app-network

Connects the cache service to the Docker network.

Step 6: Configure the Docker Volume

volumes:

mysql_data:

Creates a named Docker volume called mysql_data to store MySQL database files permanently.

Step 7: Configure the Docker Network

networks:

app-network:

Creates a custom bridge network named app-network, allowing all services to communicate securely.

Docker Compose Commands

Step 1: Start All Services

Command

docker compose up -d

Explanation:

- Reads the compose.yml file.
- Downloads required Docker images.
- Creates the containers, network, and volume.

- Starts all services in detached (background) mode.

Step 2: Check Running Containers

Command

docker compose ps

Explanation:
Displays all running containers along with their names, status, ports, and service details.

Step 3: View Logs

Command

docker compose logs

Explanation:
Displays the log output generated by all running services. It is useful for monitoring and troubleshooting the application.

Step 4: Stop and Remove Services

Command

docker compose down

Explanation:
Stops all running containers and removes the containers and Docker network created by Docker Compose.

Step 5: Remove Services and Volumes

Command

docker compose down -v

Explanation:
Stops all running containers and removes the containers, network, and Docker

volume, deleting the stored MySQL database data.

Services Used in the Experiment

Service	**Docker Image**	**Purpose**
Web	nginx:latest	Hosts the web server and serves web pages.
Application	node:18-alpine	Simulates the backend application service.
Database	mysql:8.0	Stores application data in a MySQL database.
Cache	redis:latest	Provides in-memory caching for faster data access.

Output:

compose.yml

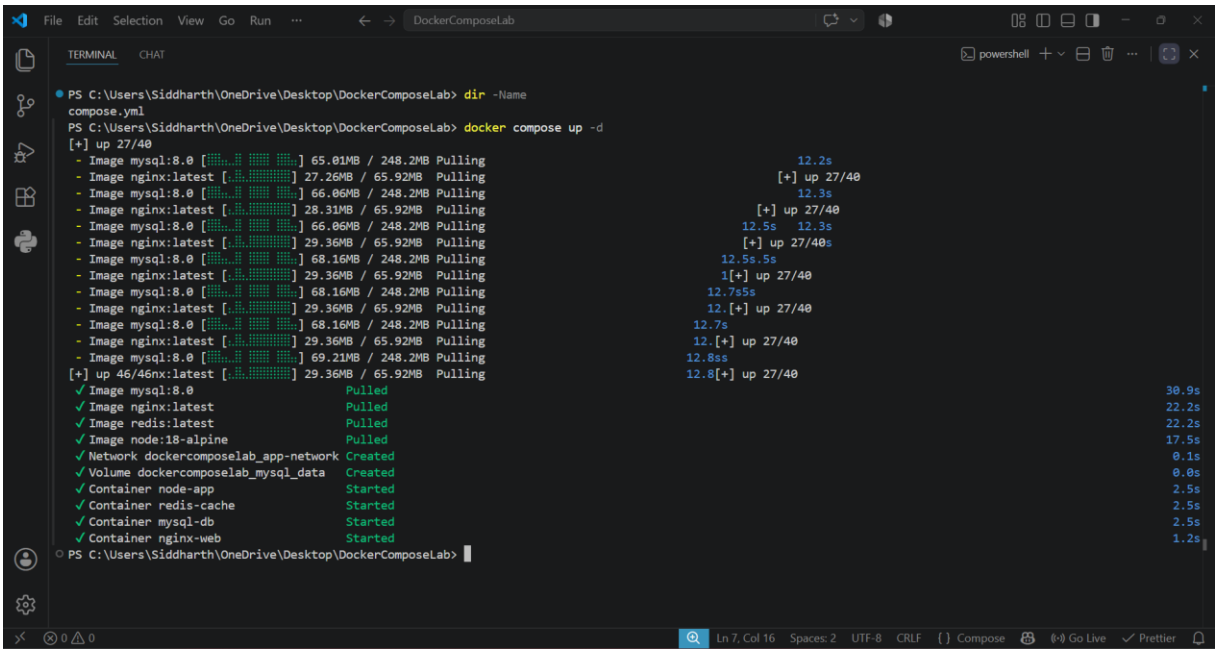
```yaml
services:
  web:
    image: nginx:latest
    container_name: nginx-web
    ports:
      - "8080:80"
    depends_on:
      - app
    networks:
      - app-network

  app:
    image: node:18-alpine
    container_name: node-app
    command: sh -c "while true; do echo 'Node.js Application Running'; sleep 30; done"
    networks:
      - app-network

  db:
    image: mysql:8.0
    container_name: mysql-db
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: mydb
    volumes:
      - mysql_data:/var/lib/mysql
    networks:
      - app-network

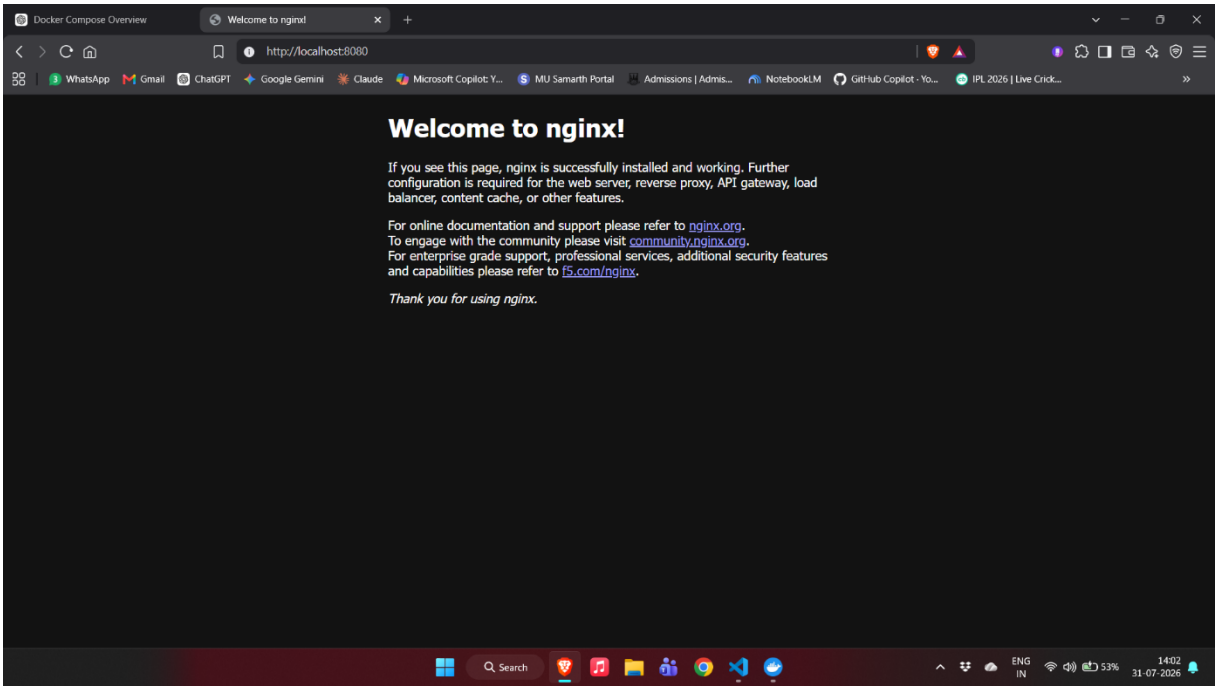
```

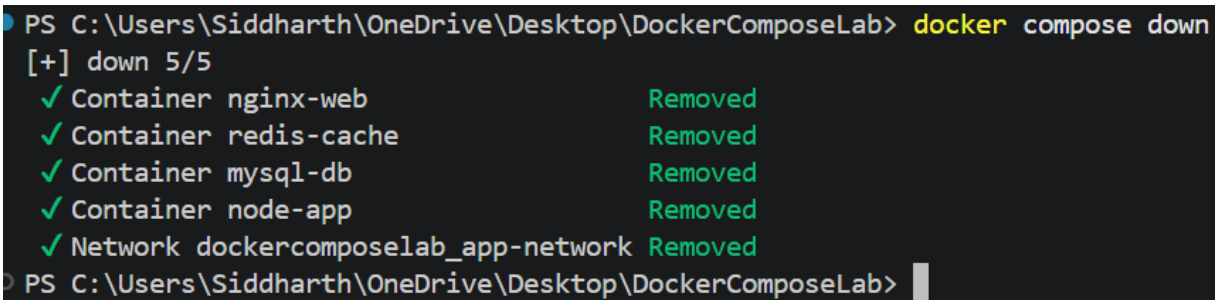
```
30      cache:
31        image: redis:latest
32        container_name: redis-cache
33        networks:
34          - app-network
35
36    volumes:
37      mysql_data:
38
39    networks:
40      app-network:
```



```
PS C:\Users\Siddharth\OneDrive\Desktop\DockerComposeLab> docker compose ps
NAME          IMAGE            COMMAND                  SERVICE   CREATED          STATUS          PORTS
mysql-db      mysql:8.0        "docker-entrypoint.s…"   db        57 seconds ago   Up 55 seconds   3306/tcp, 33060/tcp
nginx-web     nginx:latest     "/docker-entrypoint.…"   web       56 seconds ago   Up 54 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
node-app      node:18-alpine   "docker-entrypoint.s…"   app       57 seconds ago   Up 55 seconds
redis-cache   redis:latest     "docker-entrypoint.s…"   cache     57 seconds ago   Up 55 seconds   6379/tcp
PS C:\Users\Siddharth\OneDrive\Desktop\DockerComposeLab>
```

```
PS C:\Users\Siddharth\OneDrive\Desktop\DockerComposeLab> docker compose logs
redis-cache  | 1:M 31 Jul 2026 08:31:10.669 * <timeseries>      { ts-retention-policy    :           0 }
redis-cache  | 1:M 31 Jul 2026 08:31:10.669 * <timeseries>      { ts-duplicate-policy    :       block }
redis-cache  | 1:M 31 Jul 2026 08:31:10.669 * <timeseries>      { ts-chunk-size-bytes    :        4096 }
redis-cache  | 1:M 31 Jul 2026 08:31:10.669 * <timeseries>      { ts-encoding            :  compressed }
redis-cache  | 1:M 31 Jul 2026 08:31:10.669 * <timeseries>      { ts-ignore-max-time-diff:           0 }
redis-cache  | 1:M 31 Jul 2026 08:31:10.669 * <timeseries>      { ts-ignore-max-val-diff :    0.000000 }
redis-cache  | 1:M 31 Jul 2026 08:31:10.669 * <timeseries>      { ts-topology-events     :        true }
redis-cache  | 1:M 31 Jul 2026 08:31:10.669 * <timeseries> ]
redis-cache  | 1:M 31 Jul 2026 08:31:10.669 * <timeseries> Detected redis oss (cluster-enabled=no)
redis-cache  | 1:M 31 Jul 2026 08:31:10.670 * <timeseries> Subscribe to ASM events
redis-cache  | 1:M 31 Jul 2026 08:31:10.670 * <timeseries> Subscribe to topology changes events
redis-cache  | 1:M 31 Jul 2026 08:31:10.670 * <timeseries> Enabled diskless replication
redis-cache  | 1:M 31 Jul 2026 08:31:10.670 * Module 'timeseries' loaded from /usr/local/lib/redis/modules//redistimeseries.so
redis-cache  | 1:M 31 Jul 2026 08:31:10.670 * <ReJSON> Created new data type 'ReJSON-RL'
redis-cache  | 1:M 31 Jul 2026 08:31:10.670 * <ReJSON> version: 81000 git sha: unknown branch: unknown
redis-cache  | 1:M 31 Jul 2026 08:31:10.670 * <ReJSON> Exported RedisJSON_V1 API
redis-cache  | 1:M 31 Jul 2026 08:31:10.670 * <ReJSON> Exported RedisJSON_V2 API
redis-cache  | 1:M 31 Jul 2026 08:31:10.670 * <ReJSON> Exported RedisJSON_V3 API
redis-cache  | 1:M 31 Jul 2026 08:31:10.670 * <ReJSON> Exported RedisJSON_V4 API
redis-cache  | 1:M 31 Jul 2026 08:31:10.670 * <ReJSON> Exported RedisJSON_V5 API
redis-cache  | 1:M 31 Jul 2026 08:31:10.670 * <ReJSON> Exported RedisJSON_V6 API
redis-cache  | 1:M 31 Jul 2026 08:31:10.670 * <ReJSON> Exported RedisJSON_V7 API
redis-cache  | 1:M 31 Jul 2026 08:31:10.670 * <ReJSON> Exported RedisJSON_V8 API
redis-cache  | 1:M 31 Jul 2026 08:31:10.670 * <ReJSON> Enabled diskless replication
redis-cache  | 1:M 31 Jul 2026 08:31:10.670 * <ReJSON> Initialized shared string cache, thread safe: true.
redis-cache  | 1:M 31 Jul 2026 08:31:10.670 * Module 'ReJSON' loaded from /usr/local/lib/redis/modules//rejson.so
redis-cache  | 1:M 31 Jul 2026 08:31:10.671 * <search> Acquired RedisJSON_V8 API
redis-cache  | 1:M 31 Jul 2026 08:31:10.671 * Server initialized
redis-cache  | 1:M 31 Jul 2026 08:31:10.672 * Ready to accept connections tcp
redis-cache  | 1:M 31 Jul 2026 08:31:10.672 # WARNING: Redis does not require authentication and is not protected by network restrictions. Redis will accept connec
tions from any IP address on any network interface.
PS C:\Users\Siddharth\OneDrive\Desktop\DockerComposeLab>
```





Conclusion:

This experiment demonstrated how Docker Compose simplifies the deployment and management of a multi-service application using a single compose.yml file. Four services—Nginx, Node.js, MySQL, and Redis—were successfully configured and deployed. Docker Compose automatically created the required containers, network, and volume, enabling seamless communication between the services. Through this experiment, we learned how to configure, deploy, monitor, and manage multiple containerized services using Docker Compose commands, making application deployment simple, organized, and efficient.